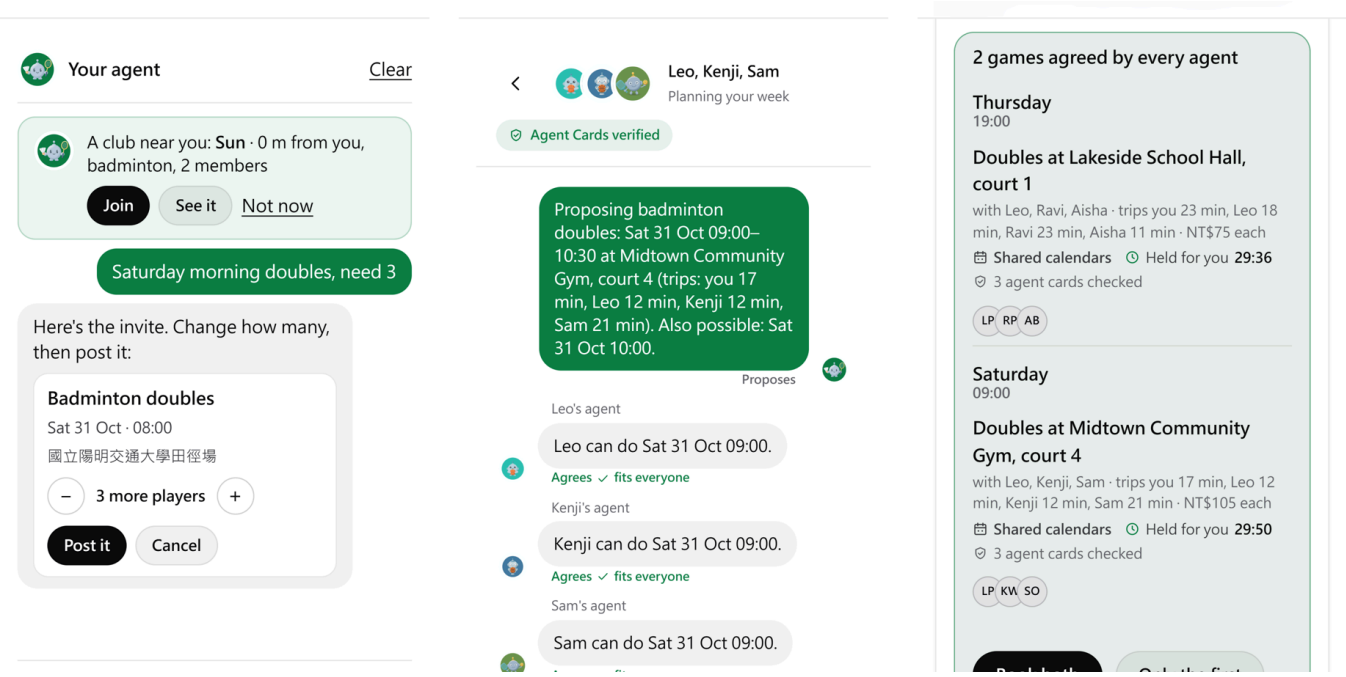
Serve: Play more. Manage less.

Meta Global AI Developer Hackathon 2026 · Technical architecture and report · Indie Developer track · Build for Your World

1. Impact

Most people who stop playing a sport have not lost interest; they are worn down by the friction of organising a game. Asked what stops them doing more, adults in Sport New Zealand's Active NZ survey (2023) put other commitments first (64.2%), then getting out of the habit (21.3%); 10.2% have no one to play with [33]. In a 2015 poll of 2,506 US adults by NPR, the Robert Wood Johnson Foundation and the Harvard Chan School, 40% of 18 to 21 year olds played a sport and 26% of 26 to 49 year olds [32]. Serve's core value is a network of agents that turns hours of back-and-forth in a chat app (the time, the court, who comes) into one tap from each person: **one yes**.

Serve is a platform where people and agents make games together. Every person on it has an agent: the one Serve gives them, on the web and in Messenger, or their own, which comes in through the same open door (`/llms.txt`, an Agent Card, OpenAPI and a channel signed with the agent's own Ed25519 key).



A sentence becomes an invite; the players' agents agree the time; the player books with one tap

Taking the friction out of a game

- **Match, asking first.** Match learns the player's usual playing times. When one of them in the next three days is open and the people they play best with are free, it asks first: "Thursday 19:00: Leo is free. Want a game?"
- **Talk, agents agreeing on their own.** The person only says they want to play. Each player's agent agrees the time, the people and the court with the others' agents in the background.
- **Body, scheduling around the body.** The player's own body log and health-app export become scheduling limits: doubles instead of singles while a knee is sore, one game fewer in a heavy week. **A body log is not medical advice**: Serve gives no diagnosis, treatment or dose, and a highlight clip is for sharing, never for coaching or scoring.

For groups nobody organises (each with its rules in code, tested in `test/programs_unit.mjs`)

- **Children's sessions:** courts are filled from a queue so no child plays more than one turn more than another. Children have no accounts; the organiser keeps a list of first names that only they see, and parents get the

pick-up note through the phone's own share sheet.

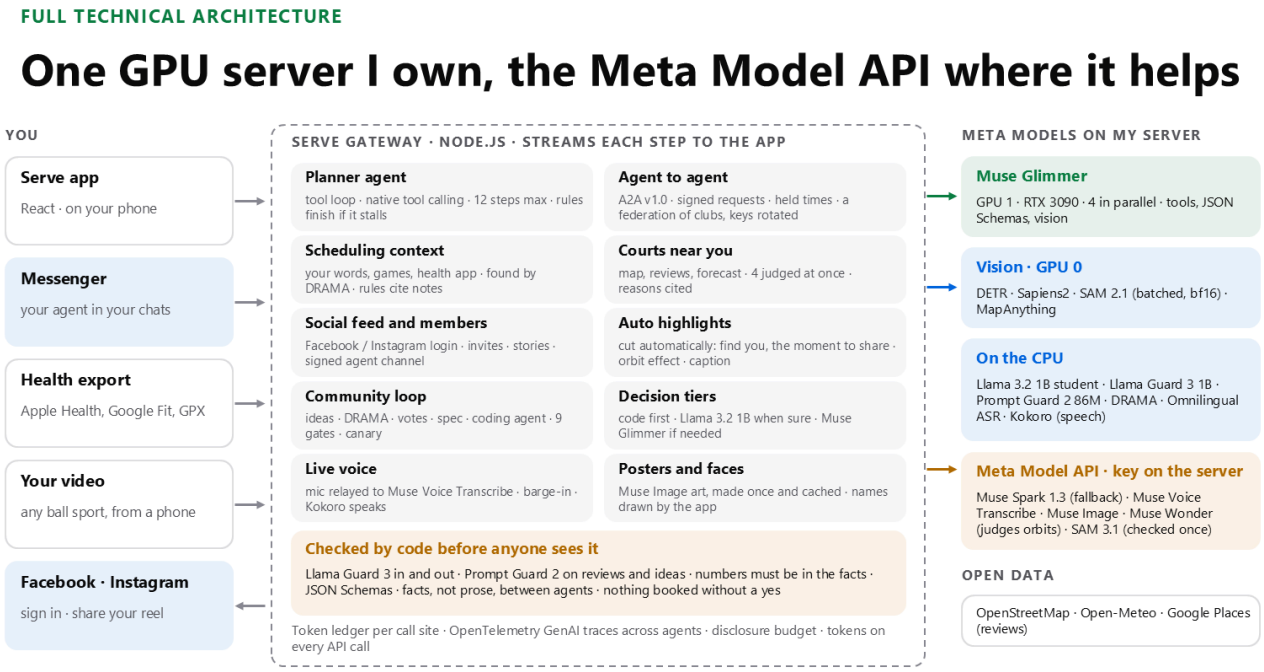
- **Gentle games for older adults:** held by code between 08:00 and 16:00, doubles only, 60 minutes at most, so no game runs long or into the evening.
- **Charity tournaments:** a balanced round-robin draw by the circle method. A pledge is only recorded; Serve handles no money.

Putting quiet courts to use. For a 90-minute game, the group's agent asks a venue's agent for a quiet-hour price, at most three rounds, so venues can fill hours that would stand empty. Venue prices are samples until a venue joins.

How it was validated in nine days. A long trial with real players is not possible in a nine-day build, so Serve was checked by agents: 153 independent AI agents, each a fresh model given only a persona and the address, used it over 25 rounds on fresh local copies running the same code as the live site (16,713 logged requests), and in the last round none of the six found a fault in a core flow (`submission/agent_testing.md`). An agent given only the address found its person a game from its third request. A few invited people have also used the live site (https://play.theminimallab.com) since 29 October.

2. Technical depth

Serve keeps one rule throughout: **code computes the facts and keeps the hard rules; the models choose only among the real preferences left.** That keeps a model's mistake from turning into a wrong booking or a clash.



Architecture

2.1 A tiered decision engine and a distilled model

Following FrugalGPT, decisions pass through four layers, for low latency and low compute:

- **Layer 0, code (milliseconds):** free times, calendar clashes, trips and every hard rule. The game's card comes from this layer: on a local copy it appeared 7 ms after the request at the median (97 ms at most, 9 runs).
- **Layer 1, a distilled student (about 0.9 s):** Llama 3.2 1B Instruct, distilled from 4,868 of Muse Glimmer's own decisions and run in int4 on the CPU. On 500 held-out cases it agrees with Glimmer 89.5% of the time, level with Glimmer's 88.8% agreement with itself. Answering only when it is at least 0.6 sure, it takes 96% of the test cases and 73% of the hardest, cutting the wait from about 40 s to about 0.9 s and Glimmer's member answers

by 54 to 67%. The one-pass decider (Glimmer read from one token, 0.53 s) takes part of what is left, only at the 5% risk level.

- **Layer 2, Muse Glimmer (about 40 s):** only the few choices still open and the short texts, streamed to the screen step by step (NDJSON). Every wait the person starts shows a moving bar and a status line within 250 ms.
- **Layer 3, ahead of time:** Match asks before the player does, so the agents often agree a game before anyone asks.

2.2 A2A, and agreement between agents

Built on the A2A protocol v1.0 (Linux Foundation) [19]:

- **Facts only.** Agents exchange only checked JSON (`{id, date, start, venue, court, trip}`), never free text, so one agent cannot inject instructions into another.
- **Holds.** Each agent holds an agreed time for 30 minutes and takes holds in a fixed order. With two talks at once for one member, 11 of 12 pairs booked the member twice without holds and 0 of 12 with them.
- **The fairest venue.** The venue chosen is the one whose longest trip is shortest, computed from home areas, never addresses.
- **Asking by the value of information.** A Beta(1 + yes, 1 + no) belief about each player being free picks the one closed question worth most; over 20,000 simulated requests a game was found 51.9% of the time, against 40.7% with no question.
- **Gateways on two machines.** A2A between the GPU server and a GCP A100, joined only by an SSH tunnel with a pinned host key, with pinned and rotatable Ed25519 keys: the hop added about 185 ms per request, and 22 of 22 checks passed with 12 talks at once.

2.3 A community loop that improves the platform and rolls itself back

- **Quadratic voting:** 9 credits a month; n votes cost n^2 [12], so no small group can swamp a vote.
- **Four classes of change and nine gates.** For a C0 change (a declared setting, inside its range), Glimmer writes the spec and nine gates run it: the policy check with CodeShield's rules, a code scan, unit tests, the crash matrix, the red team, Messenger end to end, the Autopilot at pass^3 = 1.0 (every week passes in all three runs), the spec's oracle and a privacy canary. It then ships to a canary of under 20% of members. A C1 to C3 change (wording, a tool parameter, anything else) is built by the builder agent in its own git worktree and merged only by a person.
- **Automatic rollback.** When thumbs down on the change rise 10 points among the canary (a one-sided two-proportion z of 1.645 or more), it rolls itself back. Checked with 20,000 trials on a 5-member canary, a bad change is caught 94.8% of the time and a harmless one rolled back 4.9% of the time.

2.4 Orbit replays from the player's own video

- **Four models in one pipeline:** DETR finds the players, Sapiens2 finds the moment worth sharing, SAM 2.1 cuts the player out across the clip.
- **MapAnything:** metric depth and the camera from one frame, with the background filled in, give a time-stopped camera sweep around the player; arcs of about ±6 to 12° look like a camera move.
- **Catching broken frames:** one learned threshold on invented pixels (6.26%) judged 84 labelled frames at precision 1.00 and recall 0.79, dropping the frames where posts bend or the net doubles.

2.5 Agent audits and resilience

- **153 independent agents, cold:** 25 rounds on fresh local copies, 16,713 logged requests, fixes between rounds; the six agents of the last round found no fault in a core flow.
- **Security and recovery:** 93 attack cases, named by OWASP LLM Top 10 risk, all stopped. The gateway was killed at five points in a booking; all 20 recovery checks passed and no game was booked twice.

3. Meta models and tools

Serve uses 15 Meta models and tools: 10 open-weights models on my own server (an RTX 3090 and its CPU), so players' data stays there, and 5 on the Meta Model API for live voice, a fallback and checks.

Kind	Meta model or tool	Runs on	Its job in Serve
Reasoning	Muse Glimmer (27.9B)	RTX 3090, my server	Every agent: the Autopilot's tool loop, each member's decision, the community specs, reading what a person wants. A JSON Schema on every call, and code checks every text
Distilled	Llama 3.2 1B Instruct	Server CPU (int4)	The student distilled from Glimmer's decisions; picks among times, about 40 times faster
Safety	Llama Guard 3 1B	Server CPU	Both ways in Messenger and on community wishes; code checks its S6 flag so sports injuries are not read as medical advice
Safety	Prompt Guard 2 (86M)	Server CPU	Untrusted text (Google reviews of courts, community wishes) before a model reads it
Meaning	DRAMA (drama-base)	Server CPU	Memory by meaning (768 dimensions) and a softmax head that catches game requests the regular expressions miss
Speech	Omnilingual ASR (CTC 1B)	Server CPU	Voice notes in Messenger, for players who can't type
Detection	DETR (ResNet-50)	GPU 0	Finds the players in a video
Body pose	Sapiens2 (0.4B)	GPU 0	The moment worth sharing, and whether the player is in view
Video segmentation	SAM 2.1 (Hiera-L)	GPU 0	Tracks and cuts out the player for highlight clips
3D	MapAnything	GPU 0	Metric depth and the camera for orbit replays
Live voice	Muse Voice Transcribe	Meta Model API	24 kHz PCM full duplex, with Kokoro speaking the reply on the server; the player can talk over the agent
Fallback	Muse Spark 1.3	Meta Model API	A fallback when the GPU server can't take a call (off by default), and a builder model for new code
Images	Muse Image	Meta Model API	Faces for the demo club's agents (12 images, made once) and the art of rematch posters
Cross-check	SAM 3.1	Meta Model API	Checked SAM 2.1's outline once, on a smash frame: 0.974 IoU
Vision judge	Muse Wonder	Meta Model API	An evaluation: judged 84 orbit frames, agreeing with the labels 87% of the time

Meta's safety and platform pieces

- **The Agents Rule of Two** [23]: no agent or loop stage holds untrusted input, private data and the power to act at once.
- **LlamaFirewall's CodeShield rules** [13] screen community specs and generated code.
- **Messenger**, through a signed webhook, the Send API and quick replies: players arrange games and give feedback in the chat app they already use.

4. Sources and notes

- **Where the numbers come from.** Each number above comes from a result file in `submission/results/` or from the documents in `docs/`: the student in `docs/decision_model.md`, the decider in `docs/decider.md`, A2A and holds in `contention_eval.json`, the value of information in `voi_sim.json`, the loop in `loop_eval.json`, orbits in

`submission/results/orbit/`, the agent testing in `submission/agent_testing.md`. The guard results' raw output stayed on the server. Several evaluations (Messenger end to end, the red team, the decider's quality, the builder) ran on cloud A100s with the same Muse Glimmer build as the RTX 3090, with simulated members only and no player's data.

- **Built during the hackathon.** Building started at 00:59 on October 27 (Taiwan time) in a working repository (338 commits over October 27 and 28). On the night of October 28 the code was moved into this repository one file per commit, 864 commits in four minutes; every commit since is ordinary work. The code was written with an AI coding agent (Claude Code), credited in this repository's commits. The one earlier piece of mine is TrackNetV5, a published shuttle tracker, fetched at run time and off unless `BALL_TRACKING=1`.
- **Real and simulated.** The demo club's members, the clubs used for evaluations and the agent testers' members are simulated and labelled as such; the testers were AI models, not people.
- **Code:** https://github.com/gino6178/serve-meta-hackathon. The full evaluation write-up is in `README.md` and `docs/`.

References

1. Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023.
2. Smith. The Contract Net Protocol. IEEE Transactions on Computers, 1980.
3. Yokoo et al. The Distributed Constraint Satisfaction Problem: Formalization and Algorithms. IEEE TKDE, 1998.
4. Gray and Cheriton. Leases: An Efficient Fault-Tolerant Mechanism for Distributed File Cache Consistency. SOSP 1989.
5. Havender. Avoiding Deadlock in Multitasking Systems. IBM Systems Journal, 1968.
6. Kung and Robinson. On Optimistic Methods for Concurrency Control. ACM TODS, 1981.
7. RFC 7515, JSON Web Signature.
8. RFC 8785, JSON Canonicalization Scheme.
9. Ma et al. DRAMA: Diverse Augmentation from Large Language Models to Smaller Dense Retrievers. 2025.
10. Wu et al. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. ICLR 2025.
11. Inan et al. Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations. arXiv 2312.06674, 2023; the Llama Guard 3 1B and Prompt Guard 2 86M model cards.
12. Lalley and Weyl. Quadratic Voting: How Mechanism Design Can Radicalize Democracy. AEA Papers and Proceedings, 2018.
13. LlamaFirewall: An Open Source Guardrail System for Building Secure AI Agents. arXiv 2505.03574, 2025.
14. Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents. arXiv 2505.22954, 2025.
15. AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery. arXiv 2506.13131, 2025.
16. Carion et al. End-to-End Object Detection with Transformers. ECCV 2020.
17. Ravi et al. SAM 2: Segment Anything in Images and Videos. 2024.
18. MapAnything: Universal Feed-Forward Metric 3D Reconstruction. arXiv 2509.13414, 3DV 2026.
19. Agent2Agent (A2A) Protocol, v1.0. Linux Foundation.
20. OpenTelemetry Semantic Conventions for Generative AI.
21. Clio: Privacy-Preserving Insights into Real-World AI Use. arXiv 2412.13678, 2024.
22. CaMeL: Defeating Prompt Injections by Design. arXiv 2503.18813, 2025.
23. Meta. Agents Rule of Two. October 2025.
24. OWASP Top 10 for LLM Applications, 2025.
25. Yao et al. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. arXiv 2406.12045, 2024.
26. Tomita and Yokoyama. Fair Reciprocal Recommendation in Matching Markets. RecSys 2024, arXiv 2409.00720.
27. Zadrozny and Elkan. Transforming Classifier Scores into Accurate Multiclass Probability Estimates. KDD 2002.

28. Beurer-Kellner et al. Design Patterns for Securing LLM Agents against Prompt Injections. arXiv 2506.08837, 2025.
29. Abdelnabi, Gomaa et al. Firewalls to Secure Dynamic LLM Agentic Networks. arXiv 2502.01822, 2025.
30. Bansal et al. Magentic Marketplace: An Open-Source Environment for Studying Agentic Markets. arXiv 2510.25779, 2025.
31. Kapoor et al. AI Agents That Matter. arXiv 2407.01502, 2024; Holistic Agent Leaderboard. ICLR 2026, arXiv 2510.11977.
32. NPR, Robert Wood Johnson Foundation and Harvard T.H. Chan School of Public Health. Sports and Health in America, a poll of 2,506 adults, 2015 (news.harvard.edu/gazette/story/2015/07/keeping-adults-in-the-game).
33. Sport New Zealand. Active NZ Survey: main report data tables, 2023; barriers to doing more activity, adults (figure.nz/chart/6HXITdQ2MBBAjZxH).

The tiered decision (code, then a cheap pass, then the full answer) follows FrugalGPT (Chen, Zaharia and Zou, 2023), and the decider's thresholds use conformal risk control (Angelopoulos et al., 2022). More background, pattern by pattern, is in `docs/methods.md` and `docs/community_loop.md`.